AOSP 进阶丛书 · 第九册 · ANDROID 13 (API 33)

后台与电源五系统合流

JobScheduler 十二控制器与配额计费、Alarm 的政策化定价、Doze 七态与维护窗、Standby 五桶惩罚、Power 唤醒度与 PowerGroup、Thermal 三层治理——后台经济学的完整规则书。

源码基线：AOSP Android 13 (TP1A · SDK 33) · 行号全部实测核验

目 录

前言：后台的"经济学"——时间、配额与瓦特 2
第一章 全景：五套系统的分工与合流 2
第二章 Doze：整机节流的总闸 3
第三章 JobScheduler：条件与配额的调度器 5
第四章 AlarmManager：唯一的准点源 7
第五章 App Standby：按"不常用"逐级惩罚 7
第六章 PowerManagerService：醒与睡的治理 8
第七章 Thermal：性能的反面货币 10
第八章 设计决策十问 11
附录 A 观测实操 12
附录 B 快问快答（16 条） 12
附录 C 自检题 13
附录 D 动手实验室（一天量） 13

本册速览

路径新布局：Job/Alarm/Doze 已迁 apex/jobscheduler（com.android.scheduling 实体）
Job 四站管线：入账→十二控制器与门→Quota 计费→bind 执行
Doze 七态（复刻官方 DOT 图）与维护窗递减 · QUICK_DOZE 旁路
Standby 五桶惩罚 · Power 唤醒度+PowerGroup/低功耗待机 · Thermal 三层治理
第十章串成"手机放一晚"压轴叙事；附录 D 五实验室

前言：后台的"经济学"——时间、配额与瓦特

前八册讲的都是"前台世界"（怎么启动、怎么画、怎么通信）。本册进入**后台世界**：谁能在用户看不见时继续干活、干多久、用什么代价——这套"后台经济学"由五套系统合谋：JobScheduler（条件与配额）、AlarmManager（准点）、Doze（整机节流）、App Standby（按应用惩罚）、Power/Thermal（醒睡与降速的物理约束）。

先记一条路径事实（读码第一坑）：本树（Android 13）里 JobScheduler/Alarm/Doze/AppStandby 的源码在 `frameworks/base/apex/jobscheduler/service/java/com/android/server/`（下文 JS=…/apex/jobscheduler/service/java/com/android/server）——它们就是 Mainline 模块 com.android.scheduling 的实体（书1 §16.3 表的活证据）；而 PowerManagerService/ThermalManagerService 仍在 `services/core/.../power/`。旧教材的 `services/core/.../job|alarm` 路径在本树**不存在**。

本册还有一个隐藏主线：**每一层都在把"开发者的意志"翻译成"系统的时序"**——Job 把条件翻译成窗口、Doze 把静止翻译成深睡、Standby 把冷落翻译成惩罚、Thermal 把温度翻译成降速。读完全册你会获得一种后台问题的通用分析法：先问"这件事系统想让我什么时候干"，再写代码——顺序反了就永远在对抗。

体例不变：真实源码逐行注释+调用链+图。读码路线：全景（一章）→ Doze（二章）→ Job（三章）→ Alarm（四章）→ Standby（五章）→ Power（六章）→ Thermal（七章）。

第一章 全景：五套系统的分工与合流



图 1 后台与电源全景：Job/Alarm/Doze/Standby 四调节器+Power/Thermal 两物理约束，含本树新路径提示

合流三句话（面试级浓缩）：**Alarm 管准点、Job 管条件，两者都先过"桶"与"Doze"两道闸；Power 决定整机醒睡、Thermal 决定降速降亮**。一次"每天 3 点同步"的正确姿势是 JobScheduler+约束声明（桶友好），而不是精确闹钟+前台服务保活——两种写法功耗差一个量级。

第二章 Doze：整机节流的总闸

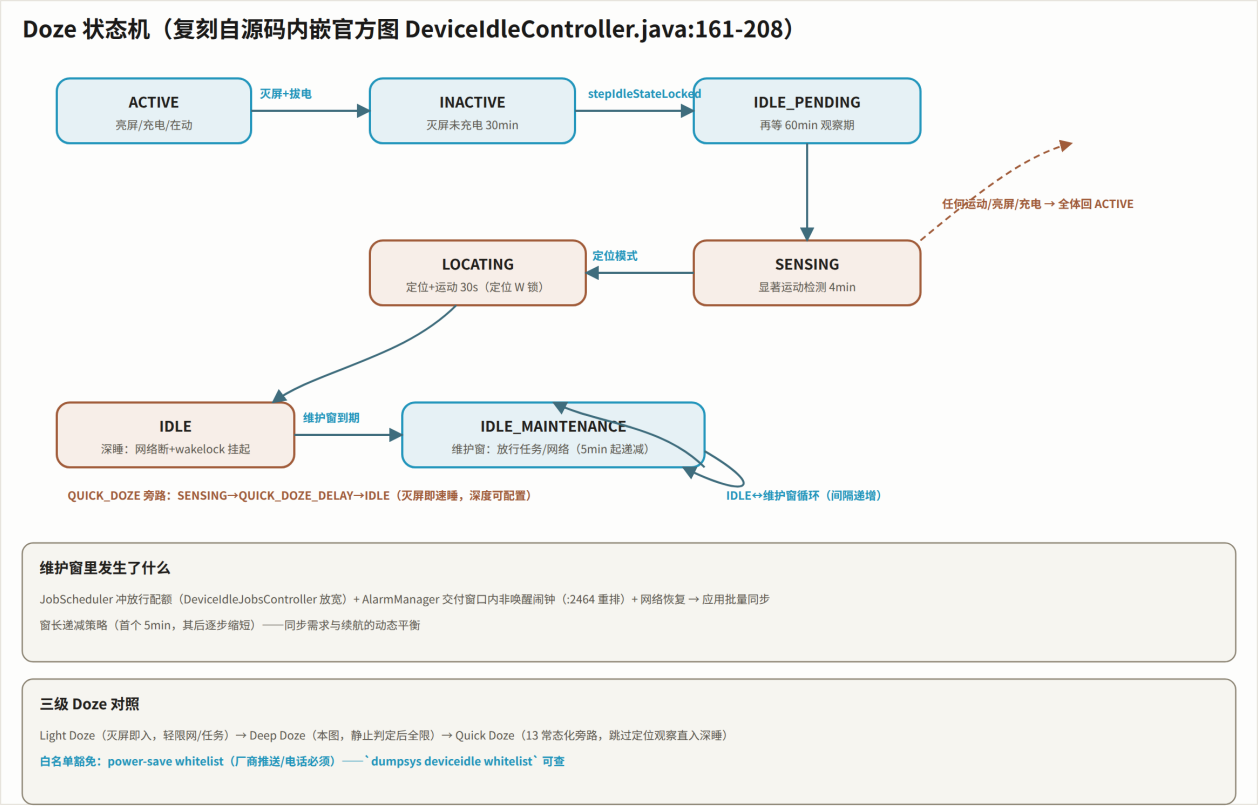


图 2 Doze 状态机（复刻自源码内嵌官方图 DeviceIdleController.java:161-208）：七态流转与 QUICK_DOZE 旁路

2.1 状态机逐段精读（真实源码骨架）

java JS/DeviceIdleController.java（骨架+讲解注释）

```
// 讲解注释①：入口三刺激：灭屏拔电（→INACTIVE，30min 计时）、
// 亮屏/充电/运动（→ACTIVE，becomeActiveIfAppropriateLocked）。
// 讲解注释②：stepIdleStateLocked 是唯一推进器（官方 DOT 图每条边都标它）：
// INACTIVE→IDLE_PENDING（再等 60min）→SENSING（显著运动检测 4min，
// 有动即回 ACTIVE——口袋里走路不睡）→LOCATING（30s 定位观察，拿 WAKE_LOCATE）
// →IDLE（深睡：网络断、wakelock 挂起、Job 冻结）
// 讲解注释③：IDLE↔IDLE_MAINTENANCE 循环：维护窗放行同步（首个约 5min，
// 之后递减）——"深睡但定期开窗"是 Doze 的核心妥协。
// 讲解注释④：QUICK_DOZE 旁路（:200 附近）：SENSING 后跳过 LOCATING
// 直入深睡——13 的默认轻量化路径（灭屏即睡）。
```

设计洞察 · 逐段精读：SENSING 为什么存在

深睡代价大（断网冻结），误判代价也大（用户正拿起手机却断网）。SENSING 用加速度计"显著运动"做**廉价的最后一问**——4 分钟无显著动才敢继续。这是"节能收益 vs 误判成本"的经典对冲，与书5 按键拦截链的"先问再分发"同构。

2.2 深睡里到底限了什么

三张清单：网络（除白名单全断）、wakelock（PARTIAL 也挂起，仅白名单例外）、Job/Alarm（非唤醒闹钟延到维护窗 `adjustDeliveryTimeBasedOnDeviceIdle:2464` 批量重排）。白名单三档：厂商预置（推送/电话）、用户加的、临时豁免（`dumpsys deviceidle` 全景）。

2.3 白名单与豁免的三层结构

text DeviceIdleController 的豁免账本（读码归纳）

```
mPowerSaveWhitelistAll // 全量白名单（厂商+用户+系统）
mPowerSaveWhitelistExceptIdle // 只豁免 Doze 不豁免省电模式（临时豁免常驻于此）
mPowerSaveWhitelistApps // 用户可见/可管理的子集（电池优化页）
// 讲解注释：Doze 中可用的=ExceptIdle+临时豁免；推送 SDK 靠厂商预置或
// 用户自弃电池优化进门——这就是"电池优化"开关的源码实体。
```

第三章 JobScheduler：条件与配额的调度器



图 3 Job 调度管线：入账→十二控制器"与"闸→Quota 计费→bind 执行，含 WorkManager 关系

3.1 提交与执行链逐段（真实锚点）

java JS/job/JobSchedulerService.java + JobServiceContext（骨架+讲解注释）

```
// 讲解注释①：JobInfo.Builder 只声明约束（网络类型/充电/空闲/最小延迟…），
// scheduleAsPackage(:1104) 入账 JobStatus；JobStore 落盘（重启不丢）。
// 讲解注释②：maybeQueueReadyJobsForExecutionLocked(:2406)：从就绪池挑队
// ——"就绪"= 十二控制器全部 satisfied（controllers/ 目录：
// Connectivity/Battery/Idle/Storage/ContentObserver/…/Quota/Restricting）。
// 讲解注释③：执行走 bindService! JobServiceContext(:79) implements
// ServiceConnection → handleServiceBoundLocked(:693/763) → onStartJob。
// 讲解注释④：bind 的意义：系统握生杀权——约束失效/超时即 unbind 收编；
// jobFinished 只是"报告完成"，不是"保活"。
```

3.2 QuotaController：执行层计费员（13 核心）

滚动窗口（10 分钟/24 小时）对每 App 记"执行时长+次数"账：前台态免费、后台按 Standby 桶打折（ACTIVE 足额 → RESTRICTED 近零）。超配额不拒绝、只排队——**把"能不能跑"变成"何时轮到"**。`dumpsys jobscheduler` 的 Job quotas 段即账本现场。WorkManager 是这层的库级封装（链式依赖/加急通道 expedited），系统视角只有 Job——"WorkManager 保活"是伪命题，保的是重试语义。

3.2.1 十二控制器速查（controllers/ 目录）

控制器	监听约束	典型卡点
ConnectivityController	网型（UNMETERED/NOT_ROAMING …）	连了计费网
BatteryController	充电中/电量阈值	拔线即取消执行
IdleController	设备空闲（Doze 联动）	深睡中反而不跑（等维护窗）
ContentObserverController	URI 变更触发	内容没变
StorageController	存储不紧张	低存储
BackgroundJobsController	App 后台资格	桶/省电剥夺
ComponentController	组件启用态	Receiver 被禁用
DeviceIdleJobsController	维护窗放宽（Doze 专属豁免）	窗内才放
PrefetchController	预取预测（13）	非预测时段
QuotaController	执行配额	账本见 §3.3
RestrictingController	桶限制	RARE 以下
TimeController	deadline 到点	最小延迟未到

读法：**每个控制器=一个"订阅了某信号的闸"**；`dumpsys jobscheduler` 的 job 详情逐控制器给出 STOPPED-REASON——排障即查表。

3.3 QuotaController 逐段（:887 isWithinQuotaLocked）

java JS/job/controllers/QuotaController.java:887 起（骨架+讲解注释）

```
boolean isWithinQuotaLocked(@NonNull final JobStatus jobStatus) {
// 讲解注释①：三重豁免：前台/充电中/Doze 维护窗内 → 免检直过
//  （:613 的调用点在"就绪→可执行"最后一闸）。
// 讲解注释②：否则按 (userId, packageName, standbyBucket) 查两本账：
// 10 分钟窗口（短时配额）与 24 小时窗口（长时配额），
// 执行时长与次数双维计费；桶越深赊账额度越低。
// 讲解注释③：超账 → 返回 false：Job 留在就绪池，等窗口滚动释放额度
// ——不是失败，是"下一班再走"。
}
```

逐段精读②的账本设计：双窗口防止两种博弈——纯短窗（攒爆发）与纯长窗（饿死近期活跃者）。**执行计费而不是排队计费**：真正跑了的才算钱，被约束卡住的不扣——鼓励把 Job 写成"快速幂等"而不是长跑。

3.4 老伙计：SyncManager 与 ContentResolver.sync

ContentResolver 的同步生态最终也汇入 Job（13 已迁 JobScheduler 驱动）：account+authority 的周期同步声明在 SyncManager 翻译成带约束的 Job。读码一景：`onPerformSync` 跑在框架的同步进程/你的 Service 里，超时与退避由 sync 框架托管——**"同步"是 Job 之上的一层领域语言**，与 WorkManager 同构（都是"把领域需求翻译成系统调度原语"）。

第四章 AlarmManager：唯一的准点源

java JS/alarm/AlarmManagerService.java:2210 起（骨架+讲解注释）

```
// 讲解注释①：setImpl(:2210)：类型（RTC/ELAPSED、wakeup 与否）+
// 窗口（window）——精确 vs 宽松的本质是"系统有多少合并余地"。
// 讲解注释②：内核层已是 timerfd（/dev/alarm 退役）：批量近邻闹钟
// 合并唤醒（省电靠合并，准点靠 wakeup 型独走）。
// 讲解注释③：Doze 交互：非 wakeup 闹钟在深睡中全部延至维护窗
//  （adjustDeliveryTimeBasedOnDeviceIdle:2464）；AllowWhileIdle
// 每 App 每 9 分钟一次的"深睡豁免"配额——高频唤醒的天敌。
```

4.1 闹钟批量（Batch）与唤醒合并

text AlarmManagerService 的节能机制（读码归纳）

```
mAlarmStore 按触发时间组织（:2626 setImplLocked 入库）
• 非 wakeup 闹钟：OS 可任意延迟交（等下一次 CPU 醒）——合并的自然土壤
• wakeup 闹钟：必须准点唤醒 CPU——近邻（同 量级 窗口）合并唤醒源，
各自 handler 仍按点投递
• Doze 中：全量非 wakeup 延至维护窗（:2464 重排），豁免账本同 Doze 白名单
// 讲解注释：所以"耗电排行里的 alarm 唤醒数"是合并后的唤醒源数，
// 不是闹钟个数——优化目标是减"源"不是减"钟"。
```

> **设计洞察 · 逐段精读：setExact 和 setAndAllowWhileIdle 的社会成本**
>
> 前者准点但阻止合并（全员准点=全员费电）；后者打穿 Doze 但有 9 分钟/App 配额。**框架用"配额"把开发者逐向宽松窗口与 Job 化**——API 设计即政策，这是读系统代码最该学的一课。

第五章 App Standby：按"不常用"逐级惩罚



图 4 App Standby 五桶（ACTIVE→RESTRICTED）：判分两路、三个下游与观测实战

五桶语义（AppStandbyController.java）：ACTIVE(10) 免费一切 → WORKING_SET(20) Job 频率约半 → FREQUENT(30) 约 1/8 → RARE(40) 几乎无后台 → RESTRICTED(45) 每日窗口级+FGS 受限。判分两路：15 天 PAR 窗口的"多久没碰"+ 13 的使用模式预测模型；**用户主动点开=一步回 ACTIVE**（:726/:767 提桶路径）。三个下游：Job 频率（Restricting/Quota）、Alarm 窗口、后台网络。实战：`am set-standby-bucket pkg restricted` 是验证"受限行为"的标准手法。

第六章 PowerManagerService：醒与睡的治理

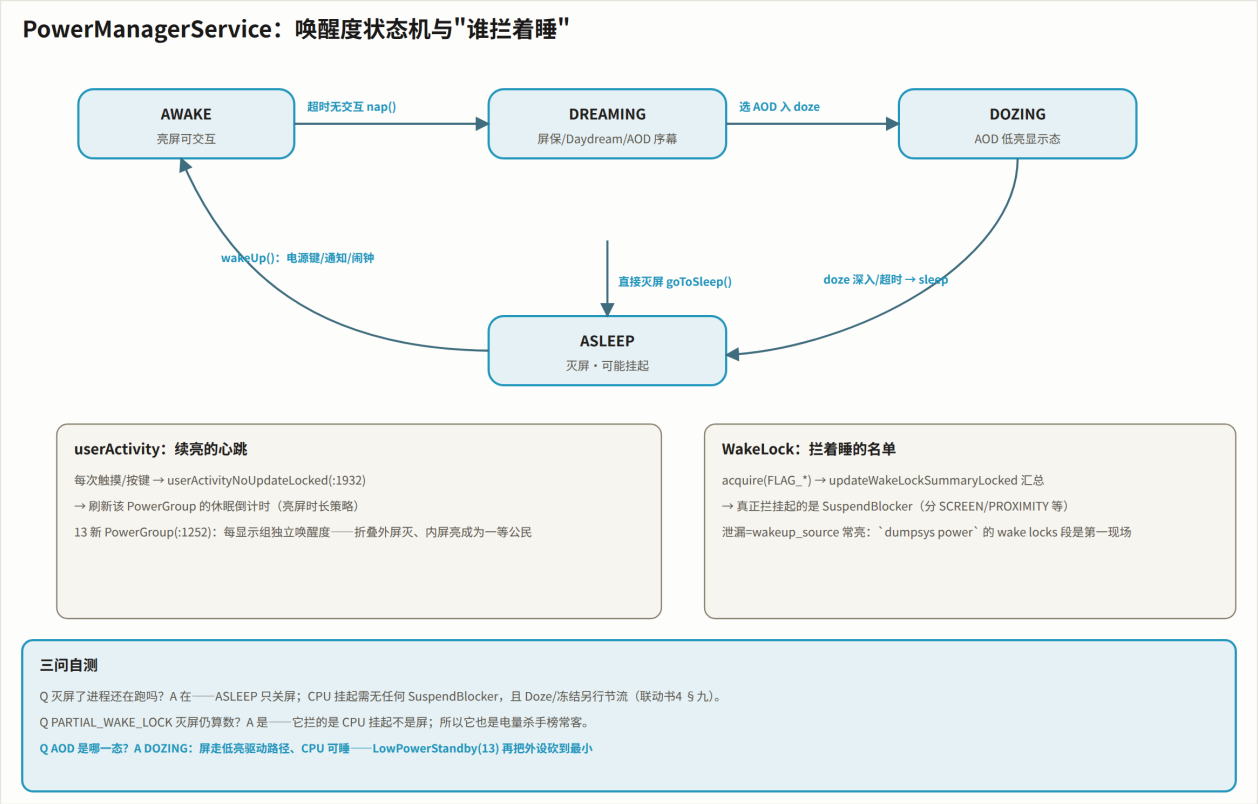


图 5 唤醒度状态机（AWAKE/DREAMING/DOZING/ASLEEP）与 PowerGroup、WakeLock→SuspendBlocker 名单

6.1 唤醒度逐段（真实锚点）

java services/core/.../power/PowerManagerService.java（骨架+讲解注释）

```
// 讲解注释①：四态 WAKEFULNESS_{AWAKE,DREAMING,DOZING,ASLEEP}
//  （import :28-31）；转移入口 wakeUpInternal/goToSleepInternal/
// napInternal，落点 setWakefulnessLocked → 广播（Notifier）+
// 显示/窗口联动（书5 的屏亮度与 doze 屏）。
// 讲解注释②：userActivityNoUpdateLocked(:1932)：触摸/按键续亮——
// 刷新该 PowerGroup 的休眠倒计时；13 新 PowerGroup(:1252)：
// 唤醒度按显示组独立（折叠双屏各自亮灭成为一等公民）。
// 讲解注释③：WakeLock 汇总（updateWakeLockSummaryLocked）→
// 真正拦 CPU 挂起的是 native SuspendBlocker（分屏/接近/常规）；
// "灭屏了进程还在跑"的真相：ASLEEP 只关屏，挂起需无人拦。
```

6.1.1 userActivity 真源码逐行（:1932-1966）

java services/core/.../power/PowerManagerService.java:1932 起（节选+讲解注释）

```
private boolean userActivityNoUpdateLocked(long eventTime, int event, int flags, int uid) {
// 讲解注释①：13 的关键变化——遍历全部 PowerGroup 分发：
// 一次触摸只续亮"事件所属显示组"，其他组不受牵连
//  （外接屏演示时手机屏可按自己的倒计时先睡）。
for (int idx = 0; idx < mPowerGroups.size(); idx++) {
if (userActivityNoUpdateLocked(mPowerGroups.valueAt(idx), ...)) ...
}
}
private boolean userActivityNoUpdateLocked(final PowerGroup powerGroup, ...) {
// 讲解注释②：合法性三查：事件时间不能早于上次睡/醒（防老事件回灌
```

```
// 把刚灭的屏又"续亮"）、systemReady 之前一律忽略。
if (eventTime < powerGroup.getLastSleepTimeLocked()
|| eventTime < powerGroup.getLastWakeTimeLocked() || !mSystemReady) {
return false;
}
// 讲解注释③：按 event 类型（TOUCH/KEY/BUTTON/OTHER）套用
// 不同的 dim/off 时长策略 → powerGroup.userActivity() 刷新倒计时。
...
// 讲解注释④：返回 needUpdate → 上层 updatePowerState() 一把梳全状态
// （唤醒度/亮度/SuspendBlocker 重算）——"记账归记账、结算一次过"。
}
```

设计洞察 · 逐段精读：为什么叫 NoUpdate

命名即架构：*NoUpdateLocked 只改账本（各 group 的倒计时），不立即触发任何状态机——统一由外层 updatePowerState 结算。**先记账后结算**让并发事件（触摸+按键同帧）合并成一次状态重算，是 PMS 全书反复出现的主模式。

6.2 LowPowerStandby（13 新）

`LowPowerStandbyController`（:71）：息屏待机低功耗态——AOD 之外的"极简醒着"（蓝牙/AOD 按配置关停），定位在"既要待机秒响应又要省电"的中间态。与 Doze 的分工：Doze 治"后台任务"，LowPowerStandby 治"息屏后的整机外设"。

6.3 省电模式（Battery Saver）家族

`power/batterysaver/` 子目录：省电模式是**策略叠加器**——不改唤醒度状态机，而是在各处消费点挂闸（亮度帽/AOD 关/CPU 频率档/动画降帧/后台限制加强），分档（off/trigger 5%/on）。与 Doze 正交：Doze 治"没人用时"，省电治"电量焦虑时"。自动触发规则（5%/15% 可配）在 `BatterySaverStateMachine`——面试常问"省电模式到底省在哪"，答案就是这张闸门清单。

第七章 Thermal：性能的反面货币

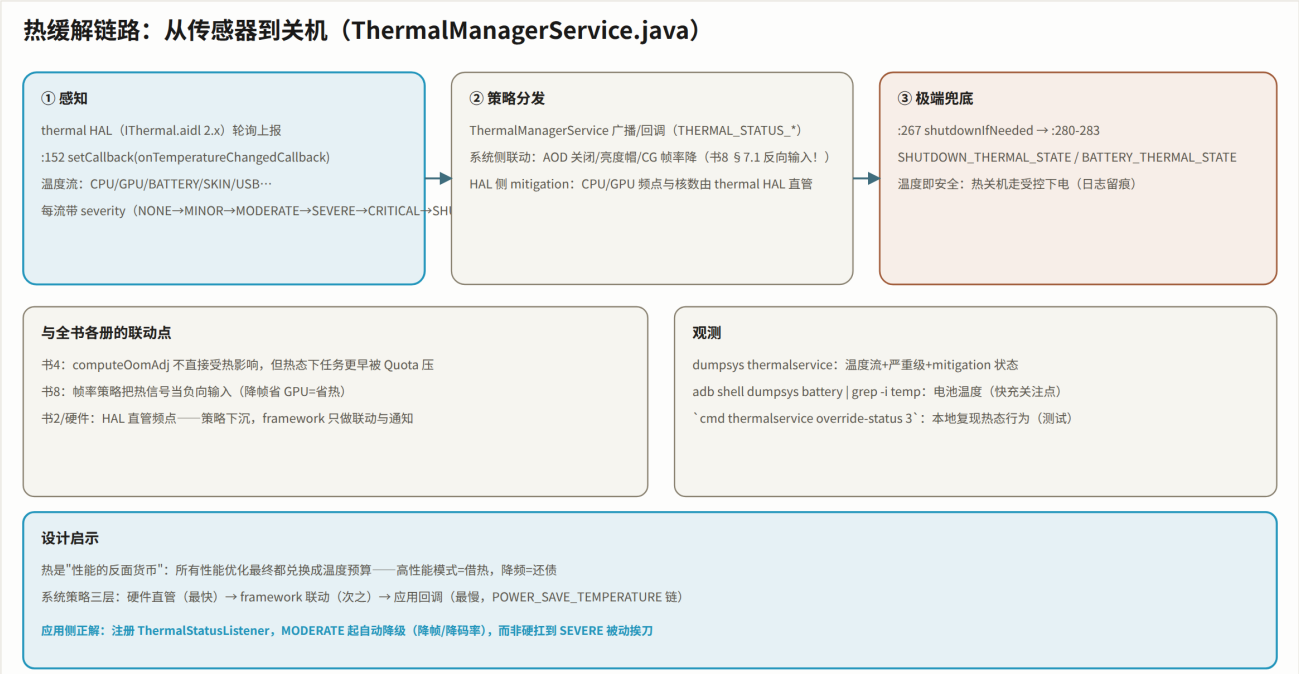


图 6 热缓解链路：HAL 感知（:152 回调）→策略联动→极端兜底（:267 关机），与全书联动点

温度流（CPU/GPU/BATTERY/SKIN…）× 严重级（NONE→SHUTDOWN 六档）。三层治理：**HAL 直管频点**（最快最底层）、framework 联动（AOD 关/亮度帽/帧率降——书8 § 7.1 的帧率策略把热当负向输入）、应用回调（THERMAL_STATUS_*，MODERATE 起自觉降级）。:267 `shutdownIfNeeded` 是终极兜底——热是安全议题。观测 `dumpsys thermalservice`；测试用 `cmd thermalservice override-status` 本地复现热态。

第八章 设计决策十问

- **为什么 Job 用 bind 而非 start?** 生杀权：约束失效即收编；start 出去的进程系统只能杀不能优雅停。
- **为什么配额是"排队"不是"拒绝"?** 后台任务是合法需求，拒绝会逼开发者对抗（保活回潮）；排队把冲突化为时序。
- **为什么 Doze 要维护窗?** 深睡不可持续（同步/推送会饿死），开窗递减是"续航与及时性"的动态帕累托。
- **为什么 SENSING 先于 IDLE?** 深睡误判代价高，加速度计是最便宜的二次确认。
- **为什么 AllowWhileIdle 限 9 分钟?** 打穿 Doze 的豁免必须稀缺，否则 Doze 名存实亡。
- **为什么五桶而不用连续分?** 离散档位可解释、可配置、可 UI 呈现（电池页"受限"）；连续分无谈判界面。
- **为什么 PowerGroup 按显示组?** 多屏时代的"这块屏的事这块屏管"，避免一屏拖全机。
- **为什么 wakelock 汇总再分层 SuspendBlocker?** 语义按用途（屏/接近/CPU），物理按是否拦挂起——两层各自演化。
- **为什么热治理 HAL 直管频点?** 热时间常数短，framework 转发太慢；策略下沉、联动上浮。

- **为什么这五套系统要分立而非合一？** 时间维度（Alarm）、条件维度（Job）、整机维度（Doze）、应用维度（Standby）、物理维度（Power/Thermal）——正交分治，各自可演进（Job/Alarm 已 Mainline 化而 Power 未必，即证据）。

附录 A 观测实操

目标	命令	看点
任务全景	dumpsys jobscheduler	约束位图/桶/配额/正在执行
闹钟全景	dumpsys alarm	每 App 闹钟数/类型/下次触发
Doze 状态	dumpsys deviceidle	当前态/白名单/维护窗统计
桶状态	am get-standby-bucket <pkg>	五桶之一
强制受限	am set-standby-bucket <pkg> restricted	行为验证
电源全景	dumpsys power	唤醒度/wakelocks/SuspendBlocker
热状态	dumpsys thermalservice	温度流+severity+mitigation

附录 B 快问快答（16 条）

- 本树 Job 源码在哪？——frameworks/base/apex/jobscheduler/（com.android.scheduling 模块实体）。
- Job 十二控制器逻辑是什么？——全部 satisfied 的"与"。
- JobService 与普通 Service 的执行差异？——bind 拉起、系统可收回。
- Quota 打折依据？——Standby 桶（前台免费）。
- WorkManager 与 JobScheduler？——库与系统服务；WM 链式/持久化，底层 Job/Alarm 双通道。
- Doze 深睡三限制？——网络断/wakelock 挂起/任务窗口化。
- 维护窗递减什么意思？——首个约 5min，后续逐次缩短（同步需求递减假设）。
- QUICK_DOZE 是什么？——跳过 LOCATING 的快速深睡旁路（13 常态）。
- 充电会退出 Doze 吗？——会（→ACTIVE）。
- AllowWhileIdle 配额？——每 App 每 9 分钟一次豁免。
- 五桶怎么快速升降？——用户点开即 ACTIVE；15 天不用渐进降桶。
- RESTRICTED 的 FGS 待遇？——受限（书4 §3.4 联动）。
- PARTIAL_WAKE_LOCK 灭屏有效吗？——有效，拦 CPU 挂起（电量杀手常客）。
- AOD 是哪个唤醒度？——DOZING（低亮驱动路径）。

- 热的 framework 动作有哪些？——AOD/亮度/帧率联动+应用回调；频点 HAL 直管。
- 学完往哪走？——把书4 冻结/书8 帧率与本册串成"功耗三件套"面试叙事。

附录 C 自检题

- 默画 Doze 七态与每条边的触发条件（含 QUICK 旁路）。
- 一个 Job 从 schedule 到 onStartJob 经过哪四站？bind 的生杀权体现在哪？
- setExact/setWindow/setAndAllowWhileIdle 三者政策含义与功耗账。
- 五桶各自砍掉什么？判分与提桶的源码路径？
- 唤醒度四态与 PowerGroup/WakeLock/SuspendBlocker 三层关系。

附录 D 动手实验室（一天量）

实验一"Doze 现场"：`dumpsys deviceidle` 记录状态；灭屏静置（或 `dumpsys deviceidle force-idle deep`）观察任务/网络停摆；触发维护窗看放行——对照第二章图逐态打卡。

实验二"Job 全链"：写最小 JobDemo（约束：充电+不限网），`dumpsys jobscheduler | grep -A5 yours` 读约束位图与配额；拔充电线看"取消原因"。

实验三"桶威力"：`am set-standby-bucket demo restricted` 后发 Job/闹钟/后台网，逐项记录行为差异；再点开 App 看一步回 ACTIVE。

实验四"wakelock 狩猎"： Overnight 静置后 `dumpsys power | grep -A3 WAKE` 与 bugreport 的 kernel wakeup_sources 对照，找出真凶并归因（本册六章 + 书4 冻结联动）。

实验五"热复现"：`cmd thermalservice override-status 3` 后观察 AOD/亮度/帧率联动（书8 §7.1 的负输入现场）。